# 10.2 Testing counter smart contract

The counter smart contract discussed in earlier chapters is a simple one, with functions to initialize, increment, decrement, and get (value). Let's `require` that this counter be allowed to hold only positive values, including `0`. To make this counter contract robust, let's add modifiers that enforce rules for incrementing and decrementing so that the positive value of the counter can be maintained. The resulting smart contract for the counter is shown in the next listing. The requirement that the value has to be positive is enforced by the modifiers added to the base code of the counter smart contract.

**Listing 10.1 Counter.sol**

```
contract Counter {
    int value;   //positive value counter

  constructor() public{
    value = 0;
  }
  modifier checkIfLessThanValue(int n) {                          ❶
    require (n <= value, 'Counter cannot become negative');
    _;
  }
  modifier checkIfNegative(int n) {                               ❶
    require (n > 0, 'Value must be greater than zero');
    _;
  }

  function get() view public returns (int){
    return value;
  }

  function initialize (int n) checkIfNegative(n) public {         ❷
    value = n;
  }
```